IoT Laboratory (A8603) – Week 1 to 12 Lab Manual

Based on the A8603 IoT Laboratory syllabus (B.Tech R22). Each week has: Aim, Components, Description, Circuit Diagram, Procedure, Program, Output and Result. 40 viva questions with answers are at the end.

0. Course Overview and Common Basics

Course outcomes (from syllabus): (1) identify hardware and software components needed for IoT applications, (2) build IoT applications with Raspberry Pi, (3) develop IoT applications on an Arduino embedded platform, (4) apply Arduino programming in IoT applications.

Tools required: Ubuntu PC, Arduino IDE, Python (Thonny), Raspberry Pi 4 board with Type-C supply, Micro-HDMI to VGA converter, 16 GB Class-10 microSD card, Arduino Uno with USB cable, breadboard, jumper wires, resistor pack, DHT11, LDR, 16x2 LCD, HC-SR04 ultrasonic sensor, LEDs, RGB LED, multimeter, soldering kit.

LED current-limiting resistor: R = (Vsupply − Vf) / I. For a red LED (Vf ≈ 2 V, I ≈ 10 mA): on Raspberry Pi 3.3 V → 130 Ω, so use 330 Ω; on Arduino 5 V → 300 Ω, so use 220 Ω–330 Ω. Never connect an LED without a resistor.

Raspberry Pi 40-pin header (pins used in these labs):

```
Pin  Function        Pin  Function
 1   3.3V             2   5V
 3   GPIO2 (SDA1)     4   5V
 5   GPIO3 (SCL1)     6   GND
 7   GPIO4            8   GPIO14 (TXD)
 9   GND             10   GPIO15 (RXD)
11   GPIO17          12   GPIO18
13   GPIO27          14   GND
15   GPIO22          16   GPIO23
17   3.3V            18   GPIO24
```

GPIO numbers follow BCM numbering (GPIO17 is physical pin 11). Raspberry Pi GPIO pins are **3.3 V only** – never feed 5 V into them.

Arduino Uno pins: D0–D13 digital (D3, D5, D6, D9, D10, D11 support PWM), A0–A5 analog inputs (10-bit ADC, values 0–1023), 5 V, 3.3 V, GND, VIN.

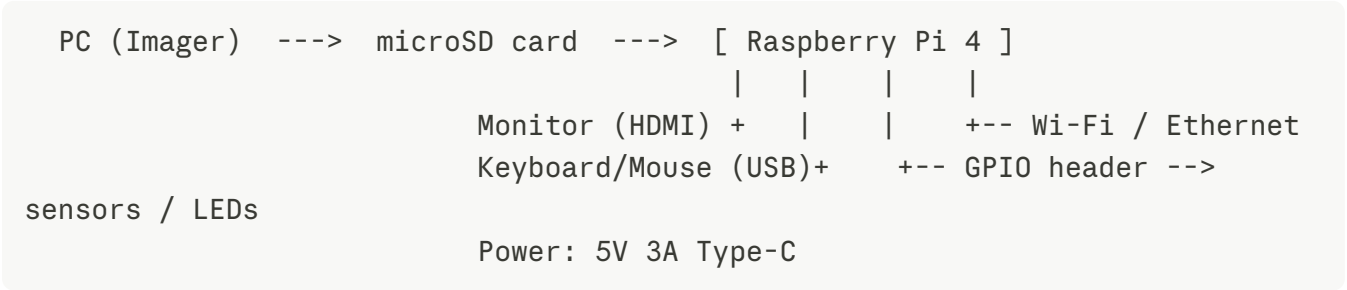
Week 1 – Study and Configure Raspberry Pi

Aim: To study the Raspberry Pi board and configure it (OS installation, first boot, network, SSH/VNC, updates).

Components: Raspberry Pi 4, 16 GB microSD, Type-C 5 V/3 A supply, monitor with micro-HDMI to VGA converter, USB keyboard and mouse, Ethernet cable or Wi-Fi, PC with card reader.

Description: The Raspberry Pi is a credit-card-sized single-board computer (SBC) with a Broadcom ARM Cortex-A72 quad-core CPU, 1/2/4/8 GB RAM, 40-pin GPIO header, USB, Ethernet, Wi-Fi, Bluetooth, HDMI, CSI camera port and DSI display port. Unlike Arduino (a microcontroller running one program with no OS), the Pi is a microprocessor board that runs a full Linux operating system (Raspberry Pi OS). The GPIO header supports digital I/O, PWM, UART (serial), SPI and I2C interfaces, which are used to connect sensors and actuators. There is no onboard ADC, so analog sensors need an external ADC chip such as MCP3008.

Block diagram:



Procedure:

1. On the PC, download and install **Raspberry Pi Imager** (`sudo apt install rpi-imager` on Ubuntu).
2. Insert the microSD card. In Imager choose Device = Raspberry Pi 4, OS = Raspberry Pi OS (32/64-bit), Storage = your SD card.
3. Click the settings (gear) option: set hostname, username/password, Wi-Fi SSID and password, and enable SSH.
4. Click Write and wait for verification. Safely eject the card.
5. Insert the card into the Pi, connect monitor, keyboard and mouse, and finally the power supply.
6. Complete the welcome wizard (country, language, time zone).
7. Open a terminal and run the commands below to update and enable interfaces.
8. Enable interfaces: `sudo raspi-config` → Interface Options → enable SSH, VNC, I2C, SPI, Serial as required.
9. Note the IP address and test remote login from the PC using `ssh pi@<ip-address>`.

Commands:

```
sudo apt update && sudo apt full-upgrade -y
hostname -I                 # shows IP address
vcgencmd measure_temp       # CPU temperature
cat /proc/cpuinfo | grep Model
pinout                      # prints the GPIO pin diagram
sudo raspi-config           # enable SSH, I2C, SPI
sudo reboot
```

Output:

```
$ hostname -I
192.168.1.25
$ vcgencmd measure_temp
temp=46.7'C
$ cat /proc/cpuinfo | grep Model
Model : Raspberry Pi 4 Model B Rev 1.4
```

Result: The Raspberry Pi was installed with Raspberry Pi OS, updated, connected to the network, and accessed both locally and remotely through SSH.

Week 2 – Study and Install Arduino IDE and Different Types of Arduino

Aim: To install the Arduino IDE on Ubuntu and study the different Arduino boards.

Components: PC with Ubuntu, Arduino Uno, USB Type-A to Type-B cable.

Description: Arduino is an open-source electronics platform built around AVR (or ARM) microcontrollers. Programs are called **sketches** and have two mandatory functions: `setup()` runs once at power-up/reset, and `loop()` runs repeatedly. The Arduino IDE compiles the sketch with avr-gcc and uploads it through the USB-serial bootloader.

Board	Microcontroller	Digital / Analog pins	Flash	Notes
Uno R3	ATmega328P, 16 MHz	14 (6 PWM) / 6	32 KB	Most common, 5 V, 2 KB SRAM
Mega 2560	ATmega2560	54 / 16	256 KB	Large projects, 4 UARTs
Nano	ATmega328P	14 / 8	32 KB	Breadboard-friendly

Board	Microcontroller	Digital / Analog pins	Flash	Notes
Leonardo	ATmega32U4	20 / 12	32 KB	Acts as USB keyboard/mouse
Due	AT91SAM3X8E (ARM Cortex-M3)	54 / 12	512 KB	3.3 V logic, 84 MHz
Pro Mini	ATmega328P	14 / 6	32 KB	No USB, very small
ESP32/ESP8266 boards	Xtensa	varies	4 MB+	Built-in Wi-Fi/Bluetooth (IoT)

Block diagram of Arduino Uno:

```
USB --> [USB-Serial chip] --> [ATmega328P] --> Digital pins D0-D13
5-12V --> [Voltage regulator]      |      --> Analog inputs A0-A5
                             [16 MHz crystal, Reset button]
```

Procedure:

1. Download Arduino IDE (AppImage or tar.xz) from arduino.cc for Linux, or use `sudo snap install arduino`.
2. Extract the archive and run `./install.sh`, then launch the IDE.
3. Give permission for USB serial access: `sudo usermod -aG dialout $USER`, then log out and log in again.
4. Connect the Arduino Uno using the USB cable.
5. Select **Tools → Board → Arduino AVR Boards → Arduino Uno**.
6. Select **Tools → Port → /dev/ttyACM0**.
7. Open **File → Examples → 01.Basics → Blink** and click **Verify** (✓) then **Upload** (→).
8. The onboard LED (pin 13) starts blinking, confirming successful installation.

Output:

```
Sketch uses 924 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for
local variables.
Done uploading.
```

The onboard LED "L" blinks once per second.

Result: Arduino IDE was installed, the Uno was detected and a test sketch was uploaded; different Arduino board types were studied.

Week 3 – Blink an LED using Raspberry Pi

Aim: To write a Python program for Raspberry Pi to blink an LED connected to a GPIO pin.

Components: Raspberry Pi, LED, 330 Ω resistor, breadboard, jumper wires.

Description: A GPIO pin configured as an output drives 3.3 V (HIGH) or 0 V (LOW). When HIGH, current flows through the resistor and LED (anode to cathode) and the LED glows. Toggling the pin with a delay produces blinking. The RPi.GPIO library is used for pin control from Python.

Circuit diagram:

```
GPIO17 (Pin 11) ----[ 330 ohm ]----(+) LED (-)---- GND (Pin 6)

Long leg of LED (anode)  -> resistor -> GPIO17
Short leg (cathode)      -> GND
```

Procedure:

1. Connect the circuit as shown on the breadboard with the Pi powered off.
2. Power on the Pi and open Thonny (or a terminal).
3. Type the program and save it as `blink.py`.
4. Run it with `python3 blink.py` and observe the LED.
5. Press Ctrl+C to stop; GPIO.cleanup() resets the pins.

Program:

```
import RPi.GPIO as GPIO
import time

LED = 17                      # BCM numbering

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(LED, GPIO.OUT)

try:
    while True:
        GPIO.output(LED, GPIO.HIGH)
        print('LED ON')
        time.sleep(1)
        GPIO.output(LED, GPIO.LOW)
        print('LED OFF')
        time.sleep(1)
```

```
except KeyboardInterrupt:
    print('Stopped')
finally:
    GPIO.cleanup()
```

Output:

```
LED ON
LED OFF
LED ON
LED OFF
^CStopped
```

The LED glows for 1 second and stays off for 1 second, repeatedly.

Result: An LED was blinked using Raspberry Pi GPIO with Python. (Alternative: the `gpiozero` library does the same with `LED(17).blink()`.)

Week 4 – Blink an LED using Arduino IDE

Aim: To write an Arduino sketch to blink an LED.

Components: Arduino Uno, LED, 220 Ω resistor, breadboard, jumper wires.

Description: `pinMode(pin, OUTPUT)` configures a pin as output, `digitalWrite(pin, HIGH/LOW)` sets 5 V or 0 V, and `delay(ms)` pauses the program. `setup()` runs once and `loop()` repeats forever.

Circuit diagram:

```
Arduino D8 ----[ 220 ohm ]----(+) LED (-)---- Arduino GND
```

Procedure:

1. Connect D8 → 220 Ω → LED anode, LED cathode → GND.
2. Open Arduino IDE, select Board = Arduino Uno and the correct Port.
3. Type the sketch, click Verify, then Upload.
4. Observe the LED blinking; change the delay values to see the effect.

Program:

```
const int ledPin = 8;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(ledPin, HIGH);
  Serial.println("LED ON");
  delay(1000);
  digitalWrite(ledPin, LOW);
  Serial.println("LED OFF");
  delay(1000);
}
```

Output (Serial Monitor at 9600 baud):

```
LED ON
LED OFF
LED ON
LED OFF
```

Result: LED blinking was implemented using Arduino Uno with a 1-second ON/OFF interval.

Week 5 – IoT Weather Monitoring System using Raspberry Pi

Aim: To read temperature and humidity with a DHT11 sensor on Raspberry Pi and upload the data to the cloud (ThingSpeak) for remote monitoring.

Components: Raspberry Pi with internet access, DHT11 sensor (3-pin module or 4-pin with 10 kΩ pull-up), jumper wires, breadboard.

Description: The DHT11 has a capacitive humidity sensor and a thermistor, and sends a 40-bit digital signal over a **single data wire** (temperature 0–50 °C ± 2 °C, humidity 20–90 % ± 5 %, sampling at most once per second). The Pi reads it through a GPIO pin, then uploads the values over Wi-Fi using HTTP to the ThingSpeak IoT cloud, which stores and plots them. This forms a complete IoT chain: **sense → process → communicate → visualise**.

Circuit diagram:

```
 DHT11            Raspberry Pi
 VCC  ---------- 3.3V (Pin 1)
 DATA ---------- GPIO4 (Pin 7)   (10k pull-up between DATA and VCC for bare
sensor)
 GND  ---------- GND  (Pin 6)

  Sensor --> Raspberry Pi --> Wi-Fi/Internet --> ThingSpeak Cloud -->
Dashboard (phone/PC)
```

Procedure:

1. Make the connections shown above with the Pi powered off.
2. Create a free account at thingspeak.com, create a **New Channel** with Field 1 = Temperature, Field 2 = Humidity, and copy the **Write API Key**.
3. Install the libraries: `sudo apt install libgpiod2 python3-pip` then `pip3 install adafruit-circuitpython-dht requests` (add `--break-system-packages` or use a virtual environment on newer Raspberry Pi OS).
4. Write the program, paste your API key and save it as `weather.py`.
5. Run `python3 weather.py`.
6. Open the channel's Private View on ThingSpeak and observe the live graphs.

Program:

```
import time
import board
import adafruit_dht
import requests

dht = adafruit_dht.DHT11(board.D4)          # GPIO4
API_KEY = 'YOUR_WRITE_API_KEY'
URL = 'https://api.thingspeak.com/update'

while True:
    try:
        t = dht.temperature
        h = dht.humidity
        print('Temperature = {} C  Humidity = {} %'.format(t, h))
        r = requests.get(URL, params={'api_key': API_KEY,
                                      'field1': t, 'field2': h}, timeout=10)
        print('ThingSpeak entry id:', r.text)
    except RuntimeError as e:
        print('Read error, retrying:', e.args[0])   # DHT reads fail
occasionally
```

```
    except KeyboardInterrupt:
        dht.exit()
        break
    time.sleep(20)       # free ThingSpeak accepts one update per 15 s
```

Output:

```
Temperature = 29 C  Humidity = 62 %
ThingSpeak entry id: 1
Temperature = 29 C  Humidity = 61 %
ThingSpeak entry id: 2
Read error, retrying: Checksum did not validate. Try again.
```

The ThingSpeak channel shows two line charts updating every 20 seconds. (An entry id of 0 means the update was rejected – check the API key or the 15-second rate limit.)

Result: An IoT weather monitoring system was built that senses temperature and humidity with DHT11 and publishes them to the cloud using Raspberry Pi.

Week 6 – Arduino Program to Monitor Temperature and Humidity using DHT11

Aim: To measure temperature and humidity using a DHT11 sensor and an Arduino Uno and display the values on the Serial Monitor.

Components: Arduino Uno, DHT11 sensor, 10 kΩ resistor (if bare sensor), jumper wires, breadboard.

Description: The DHT11 outputs a calibrated digital value; the Arduino requests data by pulling the data line low and then decodes the pulse widths. The Adafruit **DHT sensor library** (with Adafruit Unified Sensor) hides this timing work behind `readTemperature()` and `readHumidity()`. The functions return NaN on failure, which the program must check using `isnan()`. The **heat index** combines temperature and humidity to give the "feels-like" value.

Circuit diagram:

```
DHT11 VCC  ---- Arduino 5V
DHT11 DATA ---- Arduino D2   ---[10k]--- 5V (pull-up, bare sensor only)
DHT11 GND  ---- Arduino GND
```

Procedure:

1. Connect the DHT11 as shown.

2. In Arduino IDE open **Sketch → Include Library → Manage Libraries**, and install "DHT sensor library" and "Adafruit Unified Sensor" by Adafruit.
3. Select Board = Arduino Uno and the correct Port.
4. Enter the sketch, Verify and Upload.
5. Open **Tools → Serial Monitor** and set the baud rate to 9600.
6. Breathe on the sensor and observe the humidity rising.

Program:

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
  Serial.println("DHT11 Weather Monitor");
}

void loop() {
  delay(2000);                      // DHT11 needs >= 1 s between reads
  float h = dht.readHumidity();
  float t = dht.readTemperature();  // Celsius
  if (isnan(h) || isnan(t)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  float hic = dht.computeHeatIndex(t, h, false);
  Serial.print("Humidity: ");   Serial.print(h);   Serial.print(" %  ");
  Serial.print("Temperature: "); Serial.print(t);  Serial.print(" C  ");
  Serial.print("Heat index: ");  Serial.print(hic); Serial.println(" C");
}
```

Output (Serial Monitor):

```
DHT11 Weather Monitor
Humidity: 61.00 %  Temperature: 29.00 C  Heat index: 30.35 C
Humidity: 61.00 %  Temperature: 29.00 C  Heat index: 30.35 C
Humidity: 75.00 %  Temperature: 30.00 C  Heat index: 33.72 C   <- after
breathing on sensor
```

Result: Temperature and humidity were measured with DHT11 and Arduino, and displayed on the Serial Monitor.

Week 7 – Raspberry Pi based Automated Street Lighting System

Aim: To automatically switch a street light (LED) ON in darkness and OFF in daylight using an LDR and Raspberry Pi.

Components: Raspberry Pi, LDR, 1 µF capacitor, LED, 330 Ω resistor, breadboard, jumper wires.

Description: An LDR (Light Dependent Resistor) has high resistance in the dark (hundreds of kΩ–MΩ) and low resistance in light (a few kΩ). The Raspberry Pi has **no analog input**, so the LDR is read with the **RC charge-time method**: the pin is first set as output LOW to discharge the capacitor, then switched to input; the time (loop count) taken for the capacitor to charge through the LDR to the logic-HIGH threshold is measured. **Dark → high resistance → long charge time → large count.** If the count exceeds a calibrated threshold, the street light turns ON. (If you have an LDR module with a digital DO pin, simply read that pin instead.)

Circuit diagram:

```
3.3V (Pin 1) ----[ LDR ]----+---- GPIO18 (Pin 12)
                            |
                       [ 1 uF capacitor ]
                            |
                           GND

GPIO17 (Pin 11) ---[ 330 ohm ]---(+) LED (-)--- GND     (street light)
```

Procedure:

1. Build the circuit; the capacitor's negative leg goes to GND (if polarised).
2. Run a calibration first: print the count in bright light and with the LDR covered, and choose a threshold between them (the program uses 20000 as an example).
3. Save the code as `streetlight.py` and run it.
4. Cover the LDR with your hand (night): LED turns ON. Uncover (day): LED turns OFF.

Program:

```
import RPi.GPIO as GPIO
import time

LDR = 18
LED = 17
THRESHOLD = 20000        # calibrate for your LDR and capacitor
```

```
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(LED, GPIO.OUT)

def rc_time(pin):
    count = 0
    GPIO.setup(pin, GPIO.OUT)
    GPIO.output(pin, GPIO.LOW)       # discharge capacitor
    time.sleep(0.1)
    GPIO.setup(pin, GPIO.IN)         # start charging through LDR
    while GPIO.input(pin) == GPIO.LOW and count < 100000:
        count += 1
    return count

try:
    while True:
        value = rc_time(LDR)
        if value > THRESHOLD:
            GPIO.output(LED, GPIO.HIGH)
            print('Count =', value, '-> DARK  : Street light ON')
        else:
            GPIO.output(LED, GPIO.LOW)
            print('Count =', value, '-> LIGHT : Street light OFF')
        time.sleep(0.5)
except KeyboardInterrupt:
    pass
finally:
    GPIO.cleanup()
```

Output:

```
Count = 1450  -> LIGHT : Street light OFF
Count = 1380  -> LIGHT : Street light OFF
Count = 38200 -> DARK  : Street light ON
Count = 41750 -> DARK  : Street light ON
```

(Actual counts depend on the capacitor, LDR and Pi speed.)

Result: An automated street lighting system using LDR and Raspberry Pi was implemented; the LED switches automatically according to ambient light.

Week 8 – Arduino based Automated Street Lighting System

Aim: To implement an automatic street light that turns ON at night and OFF in daylight using an LDR and Arduino.

Components: Arduino Uno, LDR, 10 kΩ resistor, LED, 220 Ω resistor, breadboard, jumper wires.

Description: The LDR and a fixed 10 kΩ resistor form a **voltage divider**. The voltage at the junction is read by analog pin A0 (10-bit ADC, 0–1023 for 0–5 V). With the LDR on the 5 V side: in bright light the LDR resistance falls, so A0 voltage is high; in darkness the LDR resistance rises, so A0 voltage falls. When the reading drops below a threshold, the LED (street light) is switched ON. Vout = 5 × R_fixed / (R_LDR + R_fixed).

Circuit diagram:

```
5V ----[ LDR ]----+----[ 10k ]---- GND
                  |
                  +---- A0

D8 ----[ 220 ohm ]----(+) LED (-)---- GND
```

Procedure:

1. Connect the voltage divider to A0 and the LED to D8.
2. Upload a simple sketch that prints `analogRead(A0)` to find values in light and in dark.
3. Set the threshold between the two values (e.g. 500) in the final program.
4. Upload the program; cover the LDR to simulate night and check the LED.

Program:

```
const int ldrPin = A0;
const int ledPin = 8;
const int threshold = 500;      // calibrate: 0-1023

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int ldrValue = analogRead(ldrPin);
  Serial.print("LDR value = ");
  Serial.print(ldrValue);
  if (ldrValue < threshold) {          // dark
    digitalWrite(ledPin, HIGH);
    Serial.println("  -> NIGHT : Street light ON");
  } else {                             // bright
    digitalWrite(ledPin, LOW);
```

```
    Serial.println("  -> DAY   : Street light OFF");
  }
  delay(500);
}
```

Output (Serial Monitor):

```
LDR value = 812  -> DAY   : Street light OFF
LDR value = 790  -> DAY   : Street light OFF
LDR value = 215  -> NIGHT : Street light ON
LDR value = 180  -> NIGHT : Street light ON
```

Result: An Arduino based automatic street lighting system was implemented using an LDR sensor.

Week 9 – Arduino: Distance Measurement using Ultrasonic Sensor and Display on LCD

Aim: To measure distance with an HC-SR04 ultrasonic sensor and show it on a 16x2 LCD using Arduino.

Components: Arduino Uno, HC-SR04, 16x2 LCD (HD44780), 10 kΩ potentiometer (contrast), 220 Ω resistor, breadboard, jumper wires.

Description: The HC-SR04 sends a 40 kHz ultrasonic burst when the TRIG pin receives a 10 µs HIGH pulse. The ECHO pin then stays HIGH for as long as the sound takes to travel to the object and back. Sound speed is about 343 m/s = 0.0343 cm/µs, and since the pulse covers the distance twice: **Distance (cm) = (pulse time in µs × 0.0343) / 2**. Range is about 2 cm to 400 cm. `pulseIn()` measures the echo width. The LCD is driven in 4-bit mode using the LiquidCrystal library.

Circuit diagram:

```
HC-SR04: VCC->5V   TRIG->D9   ECHO->D10   GND->GND

16x2 LCD pin  ->  Connection
 1 VSS        ->  GND
 2 VDD        ->  5V
 3 V0         ->  Potentiometer middle pin (ends to 5V and GND)
 4 RS         ->  D12
 5 RW         ->  GND
 6 E          ->  D11
11 D4         ->  D5
12 D5         ->  D4
13 D6         ->  D3
14 D7         ->  D2
15 A (LED+)   ->  5V via 220 ohm
16 K (LED-)   ->  GND

  Sensor --TRIG/ECHO--> Arduino --4-bit data--> LCD
```

Procedure:

1. Connect the sensor and LCD as per the table. Adjust the potentiometer until the LCD shows boxes/text clearly.
2. Include the built-in `LiquidCrystal` library (no installation needed).
3. Upload the sketch.
4. Place an object at different distances in front of the sensor and note the values on LCD and Serial Monitor.

Program:

```
#include <LiquidCrystal.h>

const int trigPin = 9;
const int echoPin = 10;
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);   // RS, E, D4, D5, D6, D7

long duration;
float distance;

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  lcd.begin(16, 2);
  Serial.begin(9600);
  lcd.print("Ultrasonic Dist");
  delay(1500);
```

```
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);        // 10 us trigger pulse
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH, 30000);   // timeout 30 ms
  distance = duration * 0.0343 / 2;

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Distance:");
  lcd.setCursor(0, 1);
  lcd.print(distance);
  lcd.print(" cm");

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
  delay(500);
}
```

Output:

```
LCD row 1:  Distance:
LCD row 2:  25.40 cm

Serial Monitor:
Distance: 25.40 cm
Distance: 12.85 cm
Distance: 0.00 cm      <- no echo / out of range
```

Result: Distance was measured using an ultrasonic sensor and displayed on an LCD with Arduino.

Week 10 – Raspberry Pi: Distance Measurement using Ultrasonic Sensor and Display on LCD

Aim: To measure distance using HC-SR04 with Raspberry Pi and display it on a 16x2 I2C LCD.

Components: Raspberry Pi, HC-SR04, 16x2 LCD with PCF8574 I2C backpack, 1 kΩ and 2 kΩ resistors (voltage divider), jumper wires, breadboard.

Description: The HC-SR04 works at 5 V and its ECHO output is 5 V, which can damage the Pi's 3.3 V GPIO. A **voltage divider** (1 kΩ + 2 kΩ) reduces it: Vout = 5 × 2/(1+2) = 3.33 V. The program sends a 10 µs pulse on TRIG, timestamps the rising and falling edge of ECHO, and calculates distance = time × 34300 / 2 cm. The LCD uses the I2C bus (SDA/SCL), needing only 2 data wires; its address is usually 0x27 or 0x3F. Python is not real-time, so readings can show small jitter.

Circuit diagram:

```
HC-SR04 VCC  -> 5V (Pin 2)       HC-SR04 GND -> GND (Pin 6)
HC-SR04 TRIG -> GPIO23 (Pin 16)
HC-SR04 ECHO -> [1k] --+-- GPIO24 (Pin 18)
                       |
                     [2k]
                       |
                      GND

I2C LCD: VCC -> 5V (Pin 4)   GND -> GND (Pin 9)
         SDA -> GPIO2 (Pin 3)   SCL -> GPIO3 (Pin 5)
```

Procedure:

1. Enable I2C: `sudo raspi-config` → Interface Options → I2C → Enable.
2. Install libraries: `sudo apt install i2c-tools` and `pip3 install RPLCD smbus2`.
3. Verify the LCD address: `i2cdetect -y 1` (should show 27 or 3f).
4. Make connections (including the voltage divider on ECHO) and run the program.
5. Move an object toward and away from the sensor and observe the LCD.

Program:

```
import RPi.GPIO as GPIO
import time
from RPLCD.i2c import CharLCD

TRIG = 23
ECHO = 24

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
lcd = CharLCD('PCF8574', 0x27, cols=16, rows=2)

def get_distance():
```

```
    GPIO.output(TRIG, False)
    time.sleep(0.05)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)              # 10 us pulse
    GPIO.output(TRIG, False)

    start = time.time()
    timeout = start + 0.05
    while GPIO.input(ECHO) == 0 and start < timeout:
        start = time.time()          # last time ECHO was LOW
    stop = time.time()
    while GPIO.input(ECHO) == 1 and stop < timeout:
        stop = time.time()           # last time ECHO was HIGH

    return (stop - start) * 34300 / 2   # cm

try:
    while True:
        d = get_distance()
        print('Distance: %.1f cm' % d)
        lcd.clear()
        lcd.write_string('Distance:')
        lcd.cursor_pos = (1, 0)
        lcd.write_string('%.1f cm' % d)
        time.sleep(0.5)
except KeyboardInterrupt:
    pass
finally:
    lcd.clear()
    GPIO.cleanup()
```

Output:

```
Distance: 32.4 cm
Distance: 18.7 cm
Distance: 6.2 cm

LCD:  Distance:
      6.2 cm
```

Result: Distance measurement using an ultrasonic sensor with display on LCD was implemented using Raspberry Pi.

Week 11 – RGB LED using Raspberry Pi

Aim: To control an RGB LED using Raspberry Pi and produce different colours using PWM.

Components: Raspberry Pi, common-cathode RGB LED, three 330 Ω resistors, breadboard, jumper wires.

Description: An RGB LED contains red, green and blue LEDs in one package with four legs: R, G, B and a common leg (cathode in a common-cathode LED, anode in a common-anode LED). Mixing intensities creates other colours (R+G = yellow, G+B = cyan, R+B = magenta, R+G+B = white). The Pi has no true analog output, so brightness is controlled with **PWM (Pulse Width Modulation)**: the pin switches ON/OFF rapidly (e.g. 100 Hz) and the **duty cycle** (0–100 %) decides the apparent brightness. `GPIO.PWM(pin, freq)` creates software PWM and `ChangeDutyCycle()` changes the brightness.

Circuit diagram:

```
 GPIO17 (Pin 11) ---[330]--- R \
 GPIO27 (Pin 13) ---[330]--- G  >-- RGB LED --- Common cathode --- GND (Pin
14)
 GPIO22 (Pin 15) ---[330]--- B /

 (For a common-anode LED: connect the common leg to 3.3V and invert the duty
cycle.)
```

Procedure:

1. Identify the longest leg of the RGB LED – it is the common leg, with R on its one side and G, B on the other.
2. Connect each colour leg to its GPIO through a 330 Ω resistor and the common leg to GND.
3. Save the program as `rgb.py` and run it.
4. Observe the LED cycling through colours, then fading.

Program:

```
import RPi.GPIO as GPIO
import time

R, G, B = 17, 27, 22
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
for p in (R, G, B):
    GPIO.setup(p, GPIO.OUT)

pwm = {p: GPIO.PWM(p, 100) for p in (R, G, B)}   # 100 Hz
```

```
for p in pwm:
    pwm[p].start(0)

def set_color(r, g, b):          # values 0-100 (% duty cycle)
    pwm[R].ChangeDutyCycle(r)
    pwm[G].ChangeDutyCycle(g)
    pwm[B].ChangeDutyCycle(b)

colors = [('Red', 100, 0, 0), ('Green', 0, 100, 0), ('Blue', 0, 0, 100),
          ('Yellow', 100, 100, 0), ('Cyan', 0, 100, 100),
          ('Magenta', 100, 0, 100), ('White', 100, 100, 100)]

try:
    while True:
        for name, r, g, b in colors:
            print('Color:', name)
            set_color(r, g, b)
            time.sleep(1)
        for i in range(0, 101, 5):          # fade white in and out
            set_color(i, i, i); time.sleep(0.05)
        for i in range(100, -1, -5):
            set_color(i, i, i); time.sleep(0.05)
except KeyboardInterrupt:
    pass
finally:
    for p in pwm:
        pwm[p].stop()
    GPIO.cleanup()
```

Output:

```
Color: Red
Color: Green
Color: Blue
Color: Yellow
Color: Cyan
Color: Magenta
Color: White
```

The LED shows each colour for 1 second, then smoothly brightens and dims.

Result: RGB LED colours were produced and controlled using PWM on Raspberry Pi.

Week 12 – RGB LED using Arduino

Aim: To control an RGB LED with Arduino using PWM (`analogWrite`) and generate different colours.

Components: Arduino Uno, common-cathode RGB LED, three 220 Ω resistors, breadboard, jumper wires.

Description: Arduino has hardware PWM on pins 3, 5, 6, 9, 10 and 11 (about 490 Hz, 980 Hz on 5 and 6). `analogWrite(pin, value)` takes 0–255 (8-bit): 0 = always OFF, 255 = always ON, 127 = 50 % duty. Each of R, G and B can therefore have 256 levels, giving $256^3 \approx 16.7$ million colours. For a common-anode LED the value must be inverted (255 − value).

Circuit diagram:

```
D9  ---[220]--- R \
D10 ---[220]--- G  >-- RGB LED --- Common cathode --- GND
D11 ---[220]--- B /
```

Procedure:

1. Connect R, G and B legs to D9, D10 and D11 through 220 Ω resistors; connect the common cathode to GND.
2. Enter the sketch, select Uno and the port, and upload.
3. Observe the colour sequence and the fading effect; open Serial Monitor to see colour names.

Program:

```
const int redPin = 9, greenPin = 10, bluePin = 11;

void setColor(int r, int g, int b) {
  analogWrite(redPin, r);
  analogWrite(greenPin, g);
  analogWrite(bluePin, b);
}

void setup() {
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  Serial.println("Red");     setColor(255, 0, 0);     delay(1000);
  Serial.println("Green");   setColor(0, 255, 0);     delay(1000);
```

```
  Serial.println("Blue");    setColor(0, 0, 255);     delay(1000);
  Serial.println("Yellow");  setColor(255, 255, 0);   delay(1000);
  Serial.println("Cyan");    setColor(0, 255, 255);   delay(1000);
  Serial.println("Magenta"); setColor(255, 0, 255);   delay(1000);
  Serial.println("White");   setColor(255, 255, 255); delay(1000);

  for (int i = 0; i <= 255; i++) {      // fade in white
    setColor(i, i, i);
    delay(5);
  }
  for (int i = 255; i >= 0; i--) {      // fade out
    setColor(i, i, i);
    delay(5);
  }
}
```

Output (Serial Monitor):

```
Red
Green
Blue
Yellow
Cyan
Magenta
White
```

The LED displays each colour for 1 second, then fades white in and out.

Result: An RGB LED was controlled using Arduino PWM to create multiple colours.

Viva Questions with Answers (40)

A. IoT Fundamentals

1. **What is IoT?** A network of physical objects embedded with sensors, software and connectivity that collect and exchange data over the internet with little human intervention.
2. **List the characteristics of IoT.** Connectivity, intelligence, sensing, active engagement, scalability, dynamic and self-adapting nature, heterogeneity, and security concerns.
3. **What are the functional blocks of an IoT system?** Device, communication, services, management, security and application.

4. **What is the physical design of IoT?** The "things" (devices with sensors/actuators, processor, memory and network interface) and the protocols they use to connect to the network.
5. **What is the logical design of IoT?** The abstract view: functional blocks, communication model, communication APIs (REST/WebSocket), without hardware details.
6. **Name some applications of IoT.** Smart home, smart city (street lights), healthcare monitoring, agriculture, industrial IoT, weather monitoring, smart parking, logistics.
7. **Differentiate sensor and actuator.** A sensor converts a physical quantity (temperature, light) into an electrical signal; an actuator converts an electrical signal into physical action (LED, motor, relay).
8. **Name IoT communication protocols.** MQTT, CoAP, HTTP, Zigbee, Bluetooth LE, Wi-Fi, LoRa.
9. **What is the role of the cloud in IoT?** Storage, processing, analytics, visualisation and remote access to device data (e.g. ThingSpeak, AWS IoT).

B. Raspberry Pi

10. **What is Raspberry Pi?** A low-cost credit-card-sized single-board computer that runs Linux and has GPIO pins for hardware interfacing.
11. **Difference between Raspberry Pi and Arduino?** Pi is a microprocessor board with an OS, supports multitasking, Wi-Fi, HDMI, and runs at 3.3 V logic; Arduino is a microcontroller running one program with no OS, has analog inputs, and uses 5 V logic.
12. **Does Raspberry Pi have an analog input?** No. An external ADC such as MCP3008, or a method like RC charge time, is needed.
13. **What is GPIO?** General Purpose Input/Output – pins that software can configure as digital input or output.
14. **BCM vs BOARD numbering?** BCM uses the Broadcom GPIO number (GPIO17); BOARD uses the physical pin number on the header (pin 11).
15. **What is the operating voltage of Pi GPIO pins?** 3.3 V; 5 V inputs can damage the board.
16. **Why is a voltage divider used with HC-SR04 on Pi?** ECHO outputs 5 V, which must be reduced to about 3.3 V for the Pi.
17. **What are I2C, SPI and UART?** I2C: two-wire (SDA, SCL) multi-device bus with addresses; SPI: four-wire (MOSI, MISO, SCLK, CS) fast bus; UART: asynchronous serial with TX and RX.
18. **What does `GPIO.cleanup()` do?** Resets the used GPIO pins to input and releases resources, preventing warnings and stuck outputs.
19. **What is the purpose of the microSD card in the Pi?** It holds the operating system and files, acting as the Pi's boot drive and storage.

20. **How can you access a Pi without a monitor?** Use SSH or VNC after enabling them and finding the IP (headless setup).

C. Arduino

21. **What is Arduino?** An open-source microcontroller platform with an easy IDE and a C/C++-based language.
22. **Which microcontroller does Arduino Uno use?** ATmega328P at 16 MHz with 32 KB flash, 2 KB SRAM and 1 KB EEPROM.
23. **What are `setup()` and `loop()`?** `setup()` runs once at start for initialisation; `loop()` runs repeatedly forever.
24. **What is a bootloader?** A small program in flash that allows uploading sketches over USB serial without an external programmer.
25. **What does `pinMode()` do?** Sets a pin as INPUT, OUTPUT or INPUT_PULLUP.
26. **Difference between `digitalWrite` and `analogWrite`?** `digitalWrite` gives only HIGH/LOW; `analogWrite` gives PWM output 0–255 on PWM pins.
27. **What is the resolution of Arduino Uno's ADC?** 10-bit, so analog values range 0–1023 for 0–5 V (about 4.9 mV per step).
28. **Which Uno pins support PWM?** 3, 5, 6, 9, 10 and 11 (marked with ~).
29. **What is the use of the Serial Monitor?** To send text to and receive text from the Arduino through USB for debugging; `Serial.begin(9600)` sets the baud rate.
30. **Why must the baud rate match?** UART has no clock line; both sides must agree on the speed or data appears as garbage.

D. Sensors, LEDs and Programs

31. **What is PWM and duty cycle?** PWM switches a digital signal ON and OFF quickly; duty cycle is the percentage of time it is ON, which sets the average voltage/brightness.
32. **Why do we use a resistor with an LED?** To limit current and prevent the LED and the pin from burning out; R = (Vs − Vf)/I.
33. **How do you identify the LED anode?** The longer leg (and the flat edge of the casing marks the cathode).
34. **How does an LDR work?** Its resistance decreases as light intensity increases (photoconductivity).
35. **Why is an LDR used with a fixed resistor?** To form a voltage divider so that resistance changes become a measurable voltage.
36. **How does the DHT11 communicate?** Through a single-wire proprietary digital protocol that sends 40 bits (humidity, temperature, checksum).

37. **What are the DHT11 limits and how does it differ from DHT22?** DHT11: 0–50 °C ±2 °C, 20–90 % RH ±5 %, 1 Hz; DHT22: −40 to 80 °C ±0.5 °C, 0–100 % RH ±2–5 %, 0.5 Hz.
38. **How does an ultrasonic sensor measure distance?** It measures the echo return time of a 40 kHz pulse; distance = (time × speed of sound)/2.
39. **Why divide by 2 in the distance formula?** The measured time includes the trip to the object and back.
40. **Difference between common-cathode and common-anode RGB LEDs?** In common-cathode the shared leg goes to GND and a HIGH signal lights a colour; in common-anode it goes to VCC and a LOW signal lights a colour (inverted logic).
41. **How does the street-light system save energy?** It switches lights ON only when ambient light is below the threshold, automatically, without manual control.
42. **What is hysteresis, and why would you add it to the street-light program?** Using two thresholds (ON below one value, OFF above a higher one) prevents the light from flickering when the reading hovers around a single threshold.
43. **What is the use of the potentiometer in the LCD circuit?** It sets the voltage at V0 to adjust LCD contrast.
44. **Why use I2C LCD instead of a parallel LCD?** I2C needs only 2 signal wires (SDA, SCL) instead of 6 or more, leaving pins free for other devices.